

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

RAG & Knowledge Systems

About This Question Bank

This paper covers five core competency areas in agentic AI systems and workflow automation. Each set tests practitioner-level knowledge: not definitions, but the ability to diagnose architectural failures, select the correct design pattern, and evaluate tradeoffs in real-world agentic deployments.

5

Sets

15

Questions per Set

75

Questions Total

SET GUIDE

SET 1

Agent Architecture & Planning Patterns

Agent types, planning patterns, multi-agent orchestration, tool count, memory, checkpoint, clarification

SET 2

Workflow Automation, Triggers & Error Handling

Trigger design, batch vs event, DLQ, idempotency, error branching, schema evolution, SLA patterns

SET 3

Tool Design, MCP & API Integration

Tool grounding, error taxonomy, caching, rate limiting, security, inter-agent contracts, prompt injection

SET 4

Memory Systems, Context & Multi-Session Agents

Episodic / semantic / procedural memory, context compression, multi-tenant isolation, handoff summaries

SET 5

Safety, Governance & Production Readiness

Audit logging, canary deployment, scope enforcement, graceful degradation, proxy discrimination, governance

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question to reinforce the concept.

SET

1

Retrieval Fundamentals & Vector Search

Set 1 of 5 · 15 Questions · RAG & Knowledge Systems Question Bank

Q1. A RAG system retrieves the correct document 90% of the time but the final answers are wrong 35% of the time. What does this reveal about the system?

- A. The retrieval model needs to be improved to reach 100% accuracy.
- B. Retrieval and generation are separate failure points — even with correct retrieval, the generation model is failing to extract and use the right information from the retrieved content.**
- C. The embedding model is outdated and needs retraining.
- D. The context window is too small to fit the retrieved documents.

Correct Answer: B

Explanation

A 90% retrieval precision with 35% answer errors means generation is responsible for roughly 25–30% of failures. Common generation failure modes: the model ignores the retrieved text, misreads dense tables or structured data, or the answer is present but buried in a hard-to-parse section. Retrieval and generation must be evaluated and improved independently.

Q2. What is the key difference between sparse retrieval (e.g. BM25) and dense retrieval (e.g. embedding-based) in a RAG system?

- A. Sparse retrieval is slower; dense retrieval is faster.
- B. Sparse retrieval matches on exact keyword overlap and works well for precise term queries; dense retrieval matches on semantic similarity and captures meaning even when different words are used — each has complementary failure modes.**
- C. Sparse retrieval requires GPU infrastructure; dense retrieval does not.
- D. Dense retrieval always outperforms sparse retrieval on all query types.

Correct Answer: B

Explanation

BM25 excels when the query and document share exact vocabulary (product codes, proper nouns, technical terms). Dense retrieval excels on paraphrases, synonyms, and conceptual queries. Their failures are complementary — BM25 misses semantically related but lexically different docs; dense retrieval misses exact-match critical queries. Hybrid systems combining both consistently outperform either alone.

Q3. A user searches for 'return policy for defective items' and the retrieved documents are about 'refund procedures for damaged goods.' The semantic similarity is high. Which RAG property enabled this retrieval?

- A. BM25 keyword matching identified the overlapping terms.
- B. Dense embeddings captured the semantic relationship between 'return policy/defective' and 'refund procedures/damaged' — allowing retrieval of relevant content even without exact lexical overlap.**
- C. The chunking strategy split the documents to include both terms.
- D. A synonym table mapped 'defective' to 'damaged' and 'return' to 'refund'.

Correct Answer: B

Explanation

Dense embeddings map semantically related concepts close together in vector space regardless of surface form. 'Return' and 'refund', 'defective' and 'damaged' are near-synonyms that co-occur in similar contexts during training. This semantic generalisation is the core advantage of embedding-based retrieval over keyword systems.

Q4. Your vector database returns high cosine similarity scores for irrelevant documents. The query and documents are from the same domain but the retrieved chunks are still off-topic. What is the most likely cause?

- A. The vector database's indexing algorithm has a bug.
- B. The chunks are too large — they contain multiple topics, so the chunk embedding is an average of all topics, making it superficially similar to many queries without being specifically relevant to any one.**
- C. The embedding model was not trained on this domain.
- D. The similarity threshold is set too high.

Correct Answer: B

Explanation

Large chunks produce diluted embeddings — the vector represents the centroid of all topics in the chunk, not any specific topic. A 2000-token chunk covering pricing, returns, and shipping will have moderate similarity to queries about any of the three but high precision on none. Smaller, topically focused chunks produce embeddings that represent specific content and retrieve more precisely.

Q5. You are building a RAG system for a legal knowledge base. Queries are often very specific ('What is the notice period for contract termination under clause 12.3?'). Which retrieval strategy is most appropriate?

- A. Document-level retrieval — return the entire contract.
- B. Fine-grained chunk-level retrieval with small chunks (100–200 tokens), combined with metadata filtering by contract type and clause number to narrow the search space before semantic matching.**
- C. Summary-level retrieval — embed a one-paragraph summary of each contract.
- D. Keyword search only — legal queries are precise enough for BM25.

Correct Answer: B

Explanation

Precise legal queries need precise retrieval. Small chunks isolate specific clauses, preventing dilution from surrounding contract text. Metadata filtering (contract type, clause ID) prunes the search space before embedding similarity runs, dramatically improving precision. Document-level or summary-level retrieval buries the specific clause in too much surrounding text.

Q6. What is 'reranking' in a RAG pipeline and when does it improve results most?

- A. Reordering retrieved chunks alphabetically before passing to the generator.
- B. Running a second, more accurate (but slower) relevance model over the top-k retrieved candidates to reorder them by true relevance — most effective when the initial retrieval has high recall but imprecise ranking.**
- C. Regenerating the query in a different form and re-running retrieval.
- D. Splitting retrieved chunks into smaller pieces before passing to the LLM.

Correct Answer: B

Explanation

Reranking is a two-stage approach: fast approximate retrieval (ANN search over embeddings) fetches a candidate pool with high recall; a cross-encoder or LLM-based reranker then precisely scores query-document relevance. It's most impactful when top-k retrieval has good recall but poor precision — the right documents are in the pool but not at the top.

Q7. A RAG system uses cosine similarity to find the top-3 chunks. All three chunks are from the same paragraph in the same document. What problem does this indicate?

- A. The chunking overlap is too small.
- B. Redundant retrieval — the system is returning highly similar chunks that contain largely the same information, wasting the context window on repetition. Maximum marginal relevance (MMR) or deduplication at retrieval time addresses this.**
- C. The similarity threshold is too low, allowing low-quality matches.
- D. The embedding model is producing identical vectors for similar text.

Correct Answer: B

Explanation

When top-k retrieval returns near-duplicate chunks, the LLM receives the same information 3× in its context — a wasteful use of tokens that crowds out other relevant content. MMR retrieves chunks that are jointly relevant (covering different aspects of the query) rather than individually most similar, producing more diverse and information-rich context.

Q8. Which metric directly measures whether the retrieved chunks actually contain the information needed to answer a query?

- A. Retrieval latency.
- B. Context relevance (or context precision) — the proportion of retrieved chunks that contain information relevant to answering the query, as opposed to off-topic content that dilutes the context.**
- C. Embedding model perplexity on the query.
- D. The number of tokens in the retrieved context.

Correct Answer: B

Explanation

Context relevance/precision measures signal-to-noise in retrieval: what fraction of retrieved content is actually useful for the answer? High context relevance means the LLM's context is dense with useful information. Low context relevance means the LLM must sift through noise, increasing the chance of ignoring the key passage or hallucinating.

Q9. You switch from a general-purpose embedding model to a domain-specific one fine-tuned on your corpus. Retrieval precision improves significantly for technical queries but degrades for common English queries. Why?

- A. The fine-tuned model has fewer parameters than the general model.
- B. Fine-tuning on a narrow domain shifts the embedding space to cluster domain-specific concepts more tightly — improving domain retrieval but potentially compressing the semantic space for out-of-domain content, reducing general English query performance.**
- C. The fine-tuned model uses a smaller vocabulary.
- D. Domain-specific models require more tokens per chunk to produce accurate embeddings.

Correct Answer: B

Explanation

Domain fine-tuning optimises the embedding space for in-domain semantics at the cost of general coverage. This is the classic specialisation trade-off. The solution for mixed-domain corpora is often a hybrid: use the domain model for high-precision technical queries and a general model for broad queries, or fine-tune on a mix of domain and general text.

Q10. A RAG system performs well in testing but degrades after 3 months in production. No code changes were made. What is the most likely cause?

- A. The LLM provider updated their model.
- B. Knowledge base drift — new documents were added, old ones modified or deleted, and the index was not updated — causing stale or missing embeddings that no longer reflect the current knowledge base.**
- C. The vector database's approximate nearest-neighbour algorithm degraded.
- D. User query patterns shifted to a different distribution.

Correct Answer: B

Explanation

RAG systems require index maintenance. Documents that change without re-embedding, deleted documents that remain indexed, and new documents that aren't indexed all degrade retrieval quality over time. Production RAG systems need automated pipelines that detect document changes and trigger re-indexing — treating the vector index as a live data product, not a one-time artefact.

Q11. What is Approximate Nearest Neighbour (ANN) search and why is it used in production vector databases instead of exact search?

- A. ANN is more accurate than exact search for high-dimensional vectors.
- B. Exact nearest-neighbour search in high-dimensional spaces scales as $O(n \cdot d)$ — impractically slow for millions of vectors. ANN algorithms (HNSW, IVF, PQ) trade a small accuracy loss for orders-of-magnitude speed improvement, making production-scale retrieval feasible.**
- C. ANN search works without GPU hardware; exact search requires GPUs.
- D. ANN search is required by GDPR for privacy-preserving retrieval.

Correct Answer: B

Explanation

Brute-force exact search over millions of high-dimensional embeddings is computationally prohibitive at query time. ANN algorithms build index structures that allow approximate-but-fast lookup, typically returning the true nearest neighbours 95–99% of the time with 10–100× speedup. The accuracy loss is negligible in practice for retrieval tasks where top-1 precision isn't required.

Q12. You are choosing between HNSW and IVF as the ANN index structure for a vector database with 50 million documents. Which consideration most influences the choice?

- A. HNSW is always better for large datasets.
- B. HNSW offers higher recall and faster queries at the cost of higher memory usage — it keeps the full graph in RAM. IVF uses inverted file partitions, requiring less memory but more careful tuning. For 50M documents, memory budget and recall requirements jointly determine the right choice.**
- C. IVF requires GPUs; HNSW runs on CPU only.
- D. HNSW supports metadata filtering; IVF does not.

Correct Answer: B

Explanation

HNSW's graph structure requires the entire index to reside in memory — for 50M 1536-dim float32 vectors, that's ~300 GB RAM, often requiring specialised hardware. IVF partitions vectors into clusters and searches only the relevant partitions, dramatically reducing memory footprint at the cost of lower recall if cluster assignment is imperfect. The choice is a memory-recall trade-off at scale.

Q13. A RAG system must retrieve documents for queries in 12 languages. The documents are multilingual. Which embedding approach is most appropriate?

- A. Train a separate embedding model for each language.
- B. Use a multilingual embedding model (e.g. multilingual-e5, LaBSE) that maps semantically equivalent text in different languages to nearby vectors — enabling cross-lingual retrieval where a query in Hindi retrieves a relevant French document.**
- C. Translate all documents to English before embedding.
- D. Use BM25 with language-specific tokenisers for each language.

Correct Answer: B

Explanation

Multilingual embedding models are trained to align semantic equivalents across languages in a shared vector space. This enables cross-lingual retrieval without translation overhead. Translating documents to English loses linguistic nuance, adds latency and cost, and introduces translation errors. Language-specific BM25 cannot retrieve across language boundaries at all.

Q14. What is 'embedding drift' in a long-lived RAG system and why is it a problem?

- A. The vector database index slowly degrades in accuracy over time.
- B. When the embedding model is updated or swapped, new queries are embedded in the new model's space while existing documents remain in the old model's space — making similarity scores between old and new embeddings meaningless and degrading retrieval.**
- C. Embeddings gradually change as the model processes more queries.
- D. The distance between embeddings increases as the database grows.

Correct Answer: B

Explanation

Embedding spaces are model-specific. If the query embedding model changes (version update, model swap) while the document index was built with the previous model, query vectors and document vectors are no longer in the same semantic space — cosine similarity becomes unreliable. The full fix is re-embedding all documents with the new model. This is why embedding model versioning and re-indexing pipelines are critical in production.

Q15. You have 10 million product descriptions to index. Each description averages 800 tokens. Your embedding model has a 512-token limit. What is the correct approach?

- A. Truncate all descriptions to 512 tokens before embedding.
- B. Chunk each description into overlapping 400-token segments with 100-token overlap, embed each chunk separately, and at retrieval time aggregate or select the most relevant chunk from each product.**
- C. Summarise all descriptions to under 512 tokens using an LLM before embedding.
- D. Use a different embedding model with a longer context window.

Correct Answer: B

Explanation

Chunking with overlap is the standard approach for documents exceeding the embedding model's context limit. Overlap preserves context across chunk boundaries, preventing information loss at split points. Truncation drops the latter half of each description (often where specific features and specifications appear). Summarisation adds cost and loses searchable specifics. Longer-context embedding models are an option but don't eliminate the need for good chunking strategy.

SET

2

Chunking Strategies & Document Processing

Set 2 of 5 · 15 Questions · RAG & Knowledge Systems Question Bank

Q1. Fixed-size chunking (e.g. every 512 tokens) is the simplest strategy but has a well-known failure mode. What is it?

- A. Fixed-size chunks are too large for most embedding models.
- B. Fixed-size chunks split at arbitrary token positions, severing sentences and paragraphs mid-thought — causing the chunk boundary to fall in the middle of a key passage, with part of the passage in one chunk and the rest in another.**
- C. Fixed-size chunks produce embeddings with lower cosine similarity scores.
- D. Fixed-size chunking is too slow for large document collections.

Correct Answer: B

Explanation

Fixed-size chunking is agnostic to content structure. A sentence that starts at token 510 is split across chunks, with neither half being semantically complete. This is particularly damaging for numbered lists, multi-part definitions, and procedural steps. Semantic chunking (splitting at sentence, paragraph, or section boundaries) preserves content integrity at the cost of variable chunk sizes.

Q2. You are chunking a 50-page technical manual that has clear section headers. Which chunking strategy best preserves semantic coherence?

- A. Fixed 512-token chunks with 50-token overlap.
- B. Header-based semantic chunking — split at each section boundary (H1, H2), keeping each section as a chunk, and store the section path (e.g. '3.2 Installation > Prerequisites') as metadata for filtering and context.**
- C. Sentence-level chunking — one sentence per chunk.
- D. Paragraph-level chunking — one paragraph per chunk.

Correct Answer: B

Explanation

Document structure is semantic structure. Header-based chunking produces chunks that correspond to complete, self-contained topics. Storing the section hierarchy as metadata enables structured navigation (filter to chapter 3 before semantic search) and richer context injection (include the section path in the chunk for grounding). Sentence-level chunks lose context; paragraph-level may still split logical topics that span multiple paragraphs.

Q3. What is 'chunk overlap' and what problem does it solve?

- A. Embedding the same chunk multiple times to increase its retrieval probability.
- B. Including a small window of tokens from the preceding chunk at the start of the next chunk — preserving context across boundaries so that a passage that spans a chunk boundary remains retrievable and coherent.**
- C. Splitting large chunks into smaller overlapping sub-chunks for fine-grained retrieval.
- D. Assigning the same metadata to adjacent chunks to enable grouped retrieval.

Correct Answer: B

Explanation

Without overlap, a sentence split across a chunk boundary produces two incoherent fragments. Overlap (typically 10–20% of chunk size) ensures that context from the previous chunk carries forward, keeping boundary-spanning passages intact in at least one chunk. The trade-off is slight redundancy in the index — acceptable for the significant retrieval quality improvement.

Q4. A RAG system must handle PDFs that contain a mix of text, tables, charts, and scanned images. Standard text extraction produces garbled output for tables and empty strings for scanned pages. Which processing approach addresses all three content types?

- A. Extract text only and skip tables, charts, and scanned pages.
- B. Use a multi-modal extraction pipeline: structured table extraction (e.g. Camelot, pdfplumber) for tables, OCR for scanned pages, and chart-to-text conversion or captioning for figures — processing each content type with the appropriate tool before chunking.**
- C. Convert all PDFs to plain text using a general-purpose PDF reader.
- D. Use a vision-language model to describe every page as prose before chunking.

Correct Answer: B

Explanation

Different content types in PDFs require purpose-built extraction. Generic text extractors produce malformed table data and miss scanned content. A pipeline that routes by content type — structured extraction for tables (preserving row/column relationships), OCR for images, captioning or extraction for charts — produces clean, structured text for each type before chunking and embedding.

Q5. You are building a RAG system over a knowledge base where some documents are 500 words and others are 50,000 words. Fixed-size chunking is applied equally to all. What retrieval problem does this create?

- A. Short documents get split into too many fragments.
- B. Short documents are embedded as complete units while long documents are split into many chunks — creating an uneven retrieval landscape where short documents are over-represented in top-k results relative to their information density compared to long documents.**
- C. Long documents take longer to chunk and index.
- D. Fixed-size chunking cannot process documents under 512 tokens.

Correct Answer: B

Explanation

A 500-word document becomes 1 chunk; a 50,000-word document becomes 100 chunks. In top-k retrieval, the long document has 100× more chances to surface a relevant chunk — but each chunk is 1% of the document. Short, focused documents should be treated as atomic units; long documents benefit from structure-aware chunking that maps to logical sections rather than arbitrary size windows.

Q6. What is 'parent-child chunking' (also called hierarchical chunking) and what retrieval problem does it solve?

- A. Chunking parent and child XML elements separately for structured document retrieval.
- B. Indexing small chunks for precise retrieval but returning the larger parent chunk (e.g. the full paragraph or section) to the LLM — solving the tension between retrieval precision (needs small chunks) and generation context (needs larger passages for coherent answers).**
- C. Chunking documents at two levels: sentence-level and document-level.
- D. Using a parent embedding model to guide the chunking of child models.

Correct Answer: B

Explanation

Small chunks produce precise retrieval (the right sentence is found) but insufficient context for generation (a single sentence lacks surrounding context). Parent-child chunking uses child (small) chunks for retrieval and parent (large) chunks for LLM context. The child chunk is matched against the query; the parent chunk is passed to the LLM, giving it the surrounding context it needs for a coherent answer.

Q7. You are chunking meeting transcripts where the same topic is discussed across non-consecutive sections. Standard chunking will split the discussion into disconnected fragments. Which strategy best handles this?

- A. Increase the chunk size to 4,000 tokens to capture more discussion.
- B. Topic-based or semantic clustering chunking — group text segments by detected topic (using a topic model or LLM), creating chunks that contain all discussion of a given topic regardless of its position in the document.**
- C. Use a sliding window of 2,000 tokens with 1,000 overlap.
- D. Index the full transcript as a single document.

Correct Answer: B

Explanation

Standard positional chunking fails on topic-dispersed content — the same topic's discussion is split across multiple non-adjacent chunks. Topic-based chunking groups semantically related segments together, creating thematically coherent chunks. This is particularly valuable for meeting transcripts, interview recordings, and documents where logical flow is non-linear.

Q8. A RAG system processes code documentation. Code examples are often split mid-function by fixed-size chunking. Which specific chunking rule fixes this?

- A. Increase chunk size to 2,048 tokens to fit more code.
- B. Use syntax-aware chunking that detects code block boundaries (`` fences in Markdown or indentation levels in source code) and treats each complete function or code block as an atomic unit — never splitting within a function.**
- C. Strip all code blocks and index only prose documentation.
- D. Convert all code examples to prose descriptions before chunking.

Correct Answer: B

Explanation

Code has strict syntactic boundaries — splitting within a function produces syntax-invalid fragments that embed poorly and confuse the LLM. Syntax-aware chunking recognises code block boundaries in documentation and function/class boundaries in source files, keeping each logical unit intact. This is a domain-specific chunking rule that dramatically improves retrieval and generation quality for technical documentation.

Q9. Which metadata field is most valuable to store alongside each chunk in a product documentation RAG system?

- A. The chunk's embedding vector norm.
- B. Document ID, section title, product name, version number, and last-updated timestamp — enabling pre-filtering by product and version before semantic search, and post-retrieval citation of specific document sections.**
- C. The number of tokens in the chunk.
- D. The index position of the chunk within the document.

Correct Answer: B

Explanation

Rich metadata enables hybrid retrieval: filter by product='X' and version='2.1' first, then run semantic search within that filtered subset. Version number is critical for product documentation (a user asking about v2.1 should not receive v1.0 answers). Section title aids citation quality. Last-updated timestamp enables freshness filtering. Metadata transforms retrieval from pure semantic search into structured+semantic search.

Q10. What is the practical impact of chunk size on the trade-off between retrieval precision and generation quality in a RAG system?

- A. Larger chunks always improve both retrieval and generation.
- B. Smaller chunks improve retrieval precision (the returned chunk is more focused on the query) but may lack surrounding context for coherent generation. Larger chunks provide more context for generation but dilute the embedding and reduce retrieval precision. Optimal chunk size is task-specific.**
- C. Chunk size has no impact on retrieval precision, only generation quality.
- D. Smaller chunks are always better because they reduce token cost.

Correct Answer: B

Explanation

The chunk size trade-off is fundamental to RAG design. Retrieval precision peaks at small chunks (focused embeddings match specific queries accurately). Generation quality benefits from larger chunks (more surrounding context for coherent, complete answers). The tension is resolved by parent-child chunking or by empirically tuning chunk size on a representative evaluation set for the specific task.

Q11. You need to chunk a legal contract for RAG. The contract has numbered sections, sub-sections, and cross-references ('see Section 4.2'). Which chunking design preserves cross-reference navigability?

- A. Split at every numbered section boundary regardless of content.
- B. Chunk at the sub-section level, store the full section hierarchy and all cross-references as metadata, and resolve cross-references at retrieval time — so that when Section 4.2 is referenced in a retrieved chunk, Section 4.2's content can be automatically co-retrieved.**
- C. Chunk at the sentence level and embed cross-reference targets separately.
- D. Treat the entire contract as a single chunk to preserve all cross-references.

Correct Answer: B

Explanation

Cross-reference resolution is a RAG design challenge specific to structured documents. Storing cross-references as metadata enables 'follower retrieval': when a chunk references Section 4.2, the retrieval pipeline can automatically co-retrieve Section 4.2's content. Without this, the LLM receives a reference it cannot resolve, producing incomplete or fabricated answers about the referenced content.

Q12. What is 'semantic chunking' and how does it differ from fixed-size chunking in practice?

- A. Semantic chunking uses embeddings to identify topically coherent text segments by detecting points where consecutive sentences diverge semantically — splitting at topic transitions rather than at fixed token counts.**
- B. Semantic chunking adds semantic metadata to fixed-size chunks.
- C. Semantic chunking uses a larger embedding model to process bigger chunks.
- D. Semantic chunking is the same as sentence-level chunking.

Correct Answer: A

Explanation

Semantic chunking embeds consecutive sentences and monitors cosine similarity between adjacent sentence pairs. When similarity drops significantly, a topic transition is detected and a chunk boundary is placed. This produces variable-size chunks aligned to natural topic boundaries — better than fixed splits for retrieving complete topical discussions, at the cost of more complex implementation and variable chunk sizes.

Q13. A knowledge base contains documents in both text and HTML format. HTML documents include navigation menus, footers, and cookie banners that contaminate the text extraction. How should preprocessing address this?

- A. Embed the full HTML including boilerplate — the model will learn to ignore it.
- B. Apply HTML-aware extraction that strips tags and removes known boilerplate patterns (nav, footer, cookie banners) using a library like Trafilatura or BeautifulSoup with whitelist-based content extraction — preserving only the main article content before chunking.**
- C. Convert HTML to PDF first and then extract text.
- D. Manually clean each document before adding it to the knowledge base.

Correct Answer: B

Explanation

HTML boilerplate — navigation, footers, ads, cookie notices — pollutes embeddings with off-topic content. A 500-word article with 200 words of boilerplate produces embeddings that are 40% noise. Specialised libraries like Trafilatura use heuristics to identify main content regions and discard boilerplate, producing clean text for chunking. Manual cleaning doesn't scale; the model cannot reliably ignore structural noise.

Q14. You are designing a RAG system for a rapidly updating news knowledge base. New articles arrive every hour. What chunking and indexing architecture best handles this update frequency?

- A. Rebuild the entire index nightly.
- B. Implement an incremental indexing pipeline: new documents are chunked, embedded, and upserted into the vector store in real-time or near-real-time, with each chunk tagged with an ingestion timestamp for freshness filtering.**
- C. Cache the first 1,000 articles permanently and index only new articles weekly.
- D. Use BM25 only — it can be updated without re-embedding.

Correct Answer: B

Explanation

Real-time knowledge bases require incremental indexing — full rebuilds are too slow and costly for hourly updates. Upsert operations (insert or update) allow new chunks to enter the index immediately. Ingestion timestamps enable freshness filtering ('only retrieve articles from the last 24 hours') which is critical for news applications where recency determines relevance.

Q15. A RAG system chunks a document into 50 pieces. At retrieval time, the top-5 chunks are all from the same document, giving the LLM redundant context. Which retrieval design prevents this?

- A. Reduce the number of chunks per document.
- B. Apply a document-level diversity constraint: limit retrieval to at most 2 chunks per source document, then fill remaining top-k slots with the best chunks from different documents — ensuring the LLM context covers multiple sources.**
- C. Increase the chunk size so the document has fewer than 5 chunks.
- D. Deduplicate retrieved chunks based on cosine similarity.

Correct Answer: B

Explanation

Per-document retrieval caps enforce source diversity, preventing one highly relevant document from monopolising the LLM's context. For most queries, the best answer combines information from multiple sources — a cap of 1–2 chunks per source ensures the context window samples broadly. This is particularly important for factual queries where multiple perspectives or corroborating sources add value.

SET

3

Generation, Grounding & Hallucination Control

Set 3 of 5 · 15 Questions · RAG & Knowledge Systems Question Bank

Q1. A RAG system retrieves the correct document but the LLM still produces a hallucinated answer. Which of the following is the most likely cause?

- A. The LLM's temperature is too high.
- B. The answer exists in the retrieved context but is expressed in complex or dense language — the LLM paraphrases incorrectly or substitutes a plausible-sounding but wrong value from parametric memory when it struggles to extract the correct value from the text.**
- C. The retrieval system returned too many chunks.
- D. The LLM's context window is too large, causing attention to drift.

Correct Answer: B

Explanation

Even with the right document in context, generation hallucination occurs when the LLM misreads dense text (e.g. tables, nested conditions), substitutes a memorised near-match, or defaults to parametric memory when it cannot parse the retrieved content. Targeted grounding instructions ('Answer only from the provided context. Quote the relevant sentence if you can.') and structured source formatting reduce this failure mode.

Q2. Which system prompt instruction most directly improves groundedness in a RAG application?

- A. 'Be accurate and factual in your responses.'
- B. 'Answer the question using only the information provided in the context below. If the answer is not in the context, say exactly: "I don't have enough information to answer this." Do not use any knowledge outside the provided context.'**
- C. 'Always cite your sources when answering.'
- D. 'Think step by step before answering.'

Correct Answer: B

Explanation

Groundedness instructions require two components: a positive constraint (only use provided context) and a fallback behaviour (specific phrase for no-answer cases). Without the fallback, the model will fill gaps with parametric knowledge. The specific prescribed phrase ('I don't have enough information...') is more reliable than open-ended instructions — the model follows scripted fallbacks more consistently.

Q3. What is the 'answer faithfulness' metric in RAG evaluation and how is it measured?

- A. Whether the answer is factually correct according to a ground truth database.
- B. Whether every claim in the generated answer is supported by (entailed by) the retrieved context — measured by checking each claim against the source chunks using an NLI model or LLM judge.**
- C. Whether the answer is grammatically correct and well-written.
- D. The cosine similarity between the answer embedding and the query embedding.

Correct Answer: B

Explanation

Faithfulness measures hallucination at the claim level: for each factual statement in the answer, is there a supporting passage in the retrieved context? It's distinct from factual correctness (which checks against external ground truth) — a faithful answer stays within the retrieved context even if that context is wrong. Both faithfulness and correctness must be evaluated independently.

Q4. A RAG-generated answer is faithful to the retrieved context, but the context itself contains outdated information. The answer is wrong. Which RAG component failed?

- A. The generation component — the LLM should have used its training knowledge to correct the outdated context.
- B. The knowledge base maintenance pipeline — stale documents were not updated or flagged, causing the retrieval system to surface outdated content that the generator faithfully but incorrectly used.**
- C. The chunking strategy — outdated information should be in separate chunks.
- D. The embedding model — it should weight recent documents higher.

Correct Answer: B

Explanation

A faithful answer to stale context is a knowledge base maintenance failure, not a generation failure. The LLM correctly did what it was told (use only the context) but the context was outdated. Robust RAG systems treat document freshness as a data quality dimension: documents have expiry dates, update triggers, and version tracking. Expecting the LLM to override retrieved content with parametric knowledge undermines the core grounding design.

Q5. You want the RAG system's answers to include inline citations pointing to the specific chunk used. Which implementation approach is most reliable?

- A. Instruct the LLM to add citations from memory after generating the answer.
- B. Assign each retrieved chunk a unique ID before injection, include the IDs in the context, and instruct the LLM: 'After each claim, cite the chunk ID in brackets, e.g. [chunk_3]. Only cite IDs from the provided context.'**
- C. Add a citation step as a separate LLM call after the answer is generated.
- D. Use the retrieval similarity scores as citation confidence scores.

Correct Answer: B

Explanation

Pre-assigning chunk IDs and instructing the LLM to use them inline is the most reliable citation approach. The LLM can only cite IDs it has seen in the context — preventing fabricated citations. A post-hoc citation step asks the LLM to retrospectively attribute claims it already made, which is less reliable and more prone to hallucinated citations matching the right content to wrong IDs.

Q6. A RAG system is asked a question whose answer is not in the knowledge base. The LLM answers confidently using its parametric knowledge. Which system design prevented the correct 'I don't know' response?

- A. The retrieval threshold was too low, returning low-quality chunks.
- B. The system lacks a retrieval confidence gate — there is no mechanism to detect when retrieved chunks have low relevance to the query and trigger a no-answer fallback instead of proceeding to generation.**
- C. The LLM's temperature is too high, causing overconfident responses.
- D. The system prompt does not include a persona instruction.

Correct Answer: B

Explanation

A retrieval confidence gate inspects the similarity score of the top retrieved chunk: if it falls below a threshold (e.g. 0.7 cosine similarity), the system routes to a fallback response rather than generating from low-quality context. Without this gate, the LLM always generates — filling gaps with parametric knowledge when retrieval fails to find relevant content.

Q7. What is 'context stuffing' in RAG and why does it degrade answer quality?

- A. Retrieving too many chunks and concatenating them all into the LLM's context — producing a noisy, oversized context where the relevant passage is buried in irrelevant content, increasing the chance the model ignores or misreads the key information.**
- B. Compressing retrieved chunks to fit more into the context window.
- C. Adding metadata and citations to the context, increasing its total length.
- D. Using the LLM to expand short retrieved chunks into full paragraphs before injection.

Correct Answer: A

Explanation

Context stuffing — passing too many chunks indiscriminately — degrades the signal-to-noise ratio in the LLM's context. The model attends to all tokens but weights them unevenly; a key sentence buried among 10 irrelevant chunks may receive insufficient attention. Quality-over-quantity retrieval (fewer, more relevant chunks) consistently outperforms maximum-coverage approaches on answer accuracy.

Q8. A user asks a multi-part question: 'What is our refund policy, and does it differ for digital vs physical products?' The RAG system retrieves a single chunk about the general refund policy. The digital vs physical distinction is in a separate chunk. The answer covers only one part. Which retrieval strategy addresses this?

- A. Increase top-k to retrieve more chunks.
- B. Decompose multi-part queries into sub-questions, run retrieval independently for each sub-question, and merge the retrieved chunks before generation — ensuring each component of the question has dedicated retrieval coverage.**
- C. Ask the user to submit two separate queries.
- D. Increase chunk size to include both policy sections in one chunk.

Correct Answer: B

Explanation

Query decomposition for multi-part questions is a standard RAG enhancement. A single embedding of the full query may not retrieve content that answers all sub-questions equally well — one aspect often dominates the embedding. Sub-question decomposition (e.g. 'What is the general refund policy?' + 'Does refund policy differ for digital vs physical?') runs targeted retrieval for each, assembling comprehensive context for the generator.

Q9. What is 'generative hallucination amplification' in multi-hop RAG and why is it more dangerous than single-hop hallucination?

- A. Multi-hop queries cause the retrieval to fail more often.
- B. In multi-hop reasoning, the output of one generation step is used as input to the next retrieval or generation step — a hallucination in step 1 propagates and compounds through subsequent steps, producing a confidently wrong final answer built on a fabricated foundation.**
- C. Multi-hop RAG uses more LLM calls, increasing the total hallucination probability.
- D. Generative hallucination amplification only occurs with retrieval models, not generation models.

Correct Answer: B

Explanation

Multi-hop error propagation is one of the most dangerous failure modes in agentic RAG. If step 1 produces 'Company X was founded in 2005' (wrong — it was 2003), step 2 retrieves documents about 2005 events and step 3 generates an answer grounded in that wrong premise. Each step adds confidence. Validation gates between hops — checking intermediate answers against retrieved sources — are essential to interrupt propagation.

Q10. You are evaluating a RAG system on a dataset of 200 questions. The system achieves 88% answer correctness. What additional evaluation is necessary before claiming the system is production-ready?

- A. No additional evaluation is needed — 88% is above the industry threshold.
- B. Evaluate: (1) failure mode distribution — are the 12% errors clustered on a specific query type or document type? (2) hallucination rate on questions not in the knowledge base (out-of-scope queries). (3) performance on adversarial or edge-case queries from real users.**
- C. Test with 2,000 questions instead of 200.
- D. Evaluate retrieval precision separately from generation quality.

Correct Answer: B

Explanation

Aggregate accuracy hides dangerous failure patterns. 88% overall may mean 0% accuracy on a specific critical query type. Out-of-scope query behaviour (does the system hallucinate or correctly decline?) is a separate risk dimension. Adversarial testing reveals brittleness not visible in curated datasets. Production readiness requires understanding failure mode distribution, not just headline accuracy.

Q11. What is the 'distraction problem' in RAG generation and which prompt technique mitigates it?

- A. The LLM is distracted by irrelevant instructions in the system prompt; mitigated by simplifying the system prompt.
- B. When irrelevant but plausible-sounding chunks are included in the context, the LLM may incorporate their content into the answer even when it contradicts or is unrelated to the query — mitigated by instructing the model to explicitly identify and ignore non-relevant passages before answering.**
- C. The LLM loses track of the original query in long contexts; mitigated by repeating the query at the end of the context.
- D. The retrieval system is distracted by common words in the query; mitigated by removing stop words.

Correct Answer: B

Explanation

The distraction problem occurs when irrelevant retrieved chunks 'confuse' the generator into including off-topic information. Mitigation: instruct the model to first identify which passages are relevant to the query, cite only from those, and explicitly note when a retrieved passage is not relevant. This metacognitive step significantly reduces distraction effects compared to blind injection of all retrieved chunks.

Q12. A RAG system is used for financial reporting. A retrieved chunk contains a table with quarterly revenue figures. The LLM misreads a value from the wrong row. Which approach most directly reduces numerical extraction errors from tables?

- A. Increase the LLM's context window to see the full table.
- B. Convert tables to structured text representations (e.g. 'Q3 2024 Revenue: 42.3 Cr') or use a table-specific extraction prompt that asks the model to navigate row by column explicitly before extracting the target value.**
- C. Embed tables as images and use a vision model to extract values.
- D. Store all table values in the metadata instead of the chunk text.

Correct Answer: B

Explanation

LLMs misread tables because they process text sequentially — row-column navigation is error-prone in raw markdown or plaintext. Converting tables to explicit statements ('Q3 2024 Revenue: 42.3 Cr') eliminates the navigation problem. A step-by-step extraction prompt ('First identify the column header for the target value, then find the correct row') works when conversion is impractical. Vision models add latency and have their own extraction errors.

Q13. What is 'query-aware context compression' and what problem does it solve in RAG generation?

- A. Compressing the user's query to reduce retrieval time.
- B. Using an LLM to compress retrieved chunks to retain only the sentences relevant to the specific query before passing to the generator — reducing context noise and token cost while preserving the relevant signal.**
- C. Compressing the system prompt to leave more room for context.
- D. Using a smaller embedding model to reduce the dimensionality of context vectors.

Correct Answer: B

Explanation

Query-aware compression (e.g. LLMlingua, RECOMP) applies an LLM or lightweight model to filter retrieved chunks to only the sentences relevant to the query. A 500-token chunk may compress to 80 tokens of highly relevant content, allowing more chunks to fit in the context window at higher signal density. This trades a small amount of additional processing for significant improvement in generation quality and reduced token cost.

Q14. A user asks a question whose answer requires combining information from three separate retrieved chunks. The LLM produces an answer that only uses information from the first chunk. What is the most likely cause?

- A. The other two chunks had lower similarity scores and should not have been retrieved.
- B. The LLM is exhibiting primacy bias — attending most strongly to the first chunk in the context window and underweighting content in later chunks, a well-documented attention pattern in transformer models.**
- C. The answer from the first chunk was correct and the other chunks were redundant.
- D. The context window was too small to process all three chunks.

Correct Answer: B

Explanation

Primacy bias causes LLMs to over-rely on content at the beginning of their context. When three chunks are needed for synthesis, chunks 2 and 3 may receive insufficient attention if they appear later in the context. Mitigations: place the most important chunk nearest the question (recency), explicitly instruct 'use information from all provided passages', or use a synthesis prompt that asks the model to extract key points from each chunk before combining.

Q15. Which RAG system design best handles queries where the answer is 'the information is not in our knowledge base and should not be answered'?

- A. Rely on the LLM to recognise its own knowledge gaps.
- B. Implement a three-layer safety net: (1) retrieval confidence threshold — low score triggers no-answer path; (2) LLM groundedness check — if the answer requires knowledge not in retrieved context, produce the prescribed 'not available' response; (3) post-generation verification — flag answers that contain information not traceable to retrieved chunks.**
- C. Restrict the knowledge base to only safe topics.
- D. Use a classification model to filter queries before retrieval.

Correct Answer: B

Explanation

Out-of-scope query handling requires defence in depth: retrieval confidence detects failure to find relevant content; LLM grounding instructions handle cases where relevant content was retrieved but insufficient; post-generation verification catches LLM over-generation. Each layer catches failures the others miss. Relying on LLM self-awareness alone is insufficient — models are confidently wrong about what they don't know.

SET

4

Knowledge Base Design & Hybrid Systems

Set 4 of 5 · 15 Questions · RAG & Knowledge Systems Question Bank

Q1. You are designing a knowledge base for a B2B SaaS product with 500 customers, each with their own custom configurations and data. Which architectural design correctly handles per-customer data isolation?

- A. Store all customer data in one collection with a `customer_id` field and filter at query time.
- B. Use separate namespaces or collections per customer in the vector database, routing each query to only the customer's own namespace — providing structural isolation that prevents cross-customer data leakage regardless of filter logic.**
- C. Encrypt each customer's chunks with a unique key.
- D. Tag all chunks with a `customer_id` and rely on the LLM to ignore other customers' data.

Correct Answer: B

Explanation

Structural isolation (separate namespaces/collections) provides a hard boundary that cannot be bypassed by filter logic errors, prompt injection, or misconfigured queries. Relying on metadata filters for security is risky — a filter bug exposes all customers' data. Relying on the LLM to enforce isolation is never acceptable for security boundaries. Namespace isolation is the defence-in-depth approach.

Q2. What is 'hybrid search' in a RAG context and when does it consistently outperform pure vector search?

- A. Using two different LLMs for generation and retrieval.
- B. Combining dense vector search (semantic similarity) with sparse keyword search (BM25/TF-IDF) and merging results via a ranking fusion algorithm — consistently outperforming pure vector search on queries with specific proper nouns, codes, or exact-match terms that dense embeddings may not capture well.**
- C. Running retrieval on both the document text and its metadata simultaneously.
- D. Using a hybrid cloud and on-premises infrastructure for the knowledge base.

Correct Answer: B

Explanation

Hybrid search wins on exact-match queries (product codes, names, dates) where BM25 excels, and on semantic queries where dense retrieval excels. Reciprocal Rank Fusion (RRF) combines the two ranked lists effectively. Production benchmarks consistently show hybrid > pure dense or pure sparse across diverse query sets — particularly for enterprise knowledge bases with a mix of technical and conceptual queries.

Q3. Your enterprise knowledge base contains both structured data (product prices, inventory levels) and unstructured text (FAQs, manuals). What architecture best handles queries that need both?

- A. Convert all structured data to natural language descriptions and embed them.
- B. A multi-source retrieval architecture: semantic search over unstructured text chunks, SQL or API query over structured databases, and a query router that determines which source(s) to query based on the query type — with the LLM synthesising results from both sources.**
- C. Store structured data in the vector database alongside text chunks.
- D. Only use structured data for retrieval; use unstructured text for generation context only.

Correct Answer: B

Explanation

Structured and unstructured data require different retrieval paradigms. Embedding product prices into vectors and retrieving by semantic similarity loses precision (vector search can't filter exact price ranges). SQL/API queries on structured databases provide precise, filterable answers. A query router that sends queries to the right source, with the LLM synthesising across both, is the correct architecture for mixed-source knowledge bases.

Q4. What is a 'knowledge graph RAG' system and what type of query does it handle better than standard vector RAG?

- A. A RAG system that uses a graph database instead of a vector database for all retrieval.
- B. A system that augments vector retrieval with a knowledge graph that explicitly models entity relationships — excelling at multi-hop relational queries ('What products does Company X supply to Customer Y's subsidiary in Germany?') where vector similarity alone cannot traverse explicit relationships.**
- C. A RAG system that builds a graph of all retrieved documents for reranking.
- D. A system where the LLM builds a knowledge graph as part of its reasoning process.

Correct Answer: B

Explanation

Knowledge graph RAG excels at entity-relationship queries that require traversing explicit connections — supplier → product → customer → region chains. Vector search finds semantically similar text but cannot reliably answer questions that require following structured relationships across multiple hops. The knowledge graph handles relational navigation; vector search handles semantic similarity; their combination covers a broader query space than either alone.

Q5. A RAG system uses a knowledge base that is authoritative for your domain but 18 months out of date. An LLM with more recent training data could answer some queries correctly from parametric memory. How should the system handle this tension?

- A. Disable retrieval and rely entirely on the LLM's parametric memory.
- B. Use the knowledge base as the primary source, supplement with LLM parametric knowledge only for queries where retrieval returns below-threshold relevance, and always signal to the user which source was used — making the freshness trade-off transparent.**
- C. Update the knowledge base to be fully current before deploying.
- D. Use two separate systems: one retrieval-only, one LLM-only, and let the user choose.

Correct Answer: B

Explanation

The knowledge base is authoritative (accurate for its domain) even if dated; parametric LLM knowledge is recent but potentially inaccurate or hallucinated. A hybrid policy — retrieval-first, parametric fallback only for low-retrieval-confidence queries, with source transparency — gives users the best of both while making the limitations visible. Users can then decide whether to seek more current information.

Q6. What is 'self-querying retrieval' and what problem does it solve in RAG?

- A. The retrieval system querying itself to validate result quality.
- B. Using an LLM to decompose a natural language query into a structured query that includes both a semantic search component and explicit metadata filters — solving the problem of queries that implicitly specify constraints (e.g. 'Show me policies from Q4 2023') that pure semantic search cannot express.**
- C. A retrieval strategy where the LLM generates its own context before searching.
- D. Running retrieval multiple times with self-generated query variations to improve recall.

Correct Answer: B

Explanation

Self-querying retrieval parses the implicit constraints in a natural language query into explicit metadata filters. 'What were the refund policy changes in Q4 2023?' → semantic query ('refund policy changes') + metadata filter (date >= Oct 2023 AND date <= Dec 2023). Pure semantic search on the full query may return chronologically mixed results. Structured metadata filters applied before semantic search dramatically improve precision for time-sensitive or categorically constrained queries.

Q7. What is a 'dead retrieval path' in a RAG pipeline and how does it manifest to the end user?

- A. A retrieval path where the vector database returns no results; the LLM produces an error message.
- B. A configuration error where queries of a specific type bypass retrieval entirely — the LLM answers from parametric memory silently, with no signal to the user that retrieval was skipped, appearing as normal RAG-grounded output but without any grounding.**
- C. A slow retrieval path caused by unoptimised index settings.
- D. A retrieval path that always returns the same document regardless of the query.

Correct Answer: B

Explanation

Dead retrieval paths are silent failures — the system appears to work (the LLM generates an answer) but retrieval was never consulted. Common causes: a routing bug that misclassifies certain query types, a metadata filter that excludes all documents, or a similarity threshold so high that no chunk passes. The user receives a confidently ungrounded answer with no indication the knowledge base was bypassed. Monitoring retrieval success rate by query type is essential.

Q8. A document in your knowledge base is updated. The old version's chunks are in the vector index. New version chunks need to replace them exactly. Which operation achieves this correctly?

- A. Delete the old document's chunks and insert the new chunks in a transaction.**
- B. Add the new chunks without deleting old ones — the retrieval will naturally prefer newer, more relevant content.
- C. Update the text of each old chunk in-place without changing the embedding.
- D. Mark old chunks as 'deprecated' in metadata and filter them out at query time.

Correct Answer: A

Explanation

Document updates must delete old chunks and insert new ones atomically to prevent a mixed-version retrieval state. Option B leads to both old and new chunks being retrieved — producing contradictory information in the LLM's context. Option C is incorrect: updating text without re-embedding means the embedding no longer represents the content. Option D relies on filter discipline that can fail silently. Atomic delete-then-insert is the correct transaction pattern.

Q9. What is 'negative knowledge' in a RAG system and how should it be handled?

- A. Information that the system should never reveal — handled by content filtering.
- B. Knowledge of what is explicitly NOT true or NOT recommended — e.g. 'Do not take product X with medication Y' — which must be indexed and retrievable with the same reliability as positive facts, since omitting it from a retrieved context can be as harmful as hallucinating false information.**
- C. Low-quality documents that degrade retrieval — handled by quality filtering.
- D. Negative user feedback on retrieved results — handled by a feedback loop.

Correct Answer: B

Explanation

Negative knowledge (contraindications, prohibitions, exceptions) is functionally as important as positive knowledge in high-stakes domains (medical, legal, safety). Standard semantic retrieval may not surface 'Do not combine X and Y' when a user asks 'Is it safe to take X with Y?' — the positive framing of the query may not match the negative framing of the constraint. Explicit indexing of negative knowledge with both positive and negative query phrasings is essential.

Q10. A RAG system is deployed across three teams: Engineering (technical docs), HR (policies), Legal (contracts). Each team needs different access controls and different retrieval behaviour. Which architecture is most scalable?

- A. One shared knowledge base with one shared retrieval pipeline.
- B. A multi-tenant RAG architecture with team-specific namespaces in the vector store, team-specific chunking and metadata schemas tailored to each document type, shared retrieval infrastructure, and a routing layer that enforces access control before retrieval.**
- C. Three entirely separate RAG systems with no shared components.
- D. One shared knowledge base with post-retrieval access control filtering.

Correct Answer: B

Explanation

Multi-tenant RAG balances isolation and efficiency: namespaces provide data separation and access control; shared infrastructure reduces operational overhead; team-specific schemas allow tailored chunking (code-aware for engineering, section-based for legal). Post-retrieval filtering is risky (data is retrieved before access is checked). Entirely separate systems are expensive and hard to maintain. The routing + namespace approach scales cleanly.

Q11. What is 'retrieval augmented fine-tuning' (RAFT) and how does it differ from standard RAG?

- A. Fine-tuning the embedding model on domain-specific query-document pairs.
- B. Fine-tuning the LLM on examples that include retrieved documents as context — teaching the model to extract answers from retrieved passages, reason over them, and cite sources; producing a model that is both better at using retrieved context and more resistant to distraction from irrelevant passages.**
- C. A technique that automatically fine-tunes the retrieval model based on user feedback.
- D. Training the retrieval and generation components jointly end-to-end.

Correct Answer: B

Explanation

RAFT fine-tunes the generator specifically for RAG usage — training it on (query, retrieved_docs, answer) triples including 'distractor' documents it should ignore. Standard RAG uses a generic LLM that was not trained for this context structure. RAFT-trained models are significantly better at: citing specific passages, ignoring irrelevant retrieved content, and producing grounded answers — at the cost of a fine-tuning investment.

Q12. What is a 'knowledge base poisoning attack' in a RAG system and what is the most effective mitigation?

- A. A DDoS attack on the vector database server; mitigated by rate limiting.
- B. An attacker inserting malicious documents into the knowledge base that contain adversarial instructions designed to be retrieved and cause the LLM to behave maliciously — mitigated by document validation, source authentication, and treating all retrieved content as untrusted data in the generation prompt.**
- C. A data corruption attack that modifies embeddings in the vector database; mitigated by checksums.
- D. An attack that floods the knowledge base with irrelevant documents to degrade retrieval quality; mitigated by relevance scoring.

Correct Answer: B

Explanation

Knowledge base poisoning is a RAG-specific attack vector: malicious content injected into the knowledge base is retrieved and passed to the LLM as context, potentially hijacking its behaviour. Mitigations operate at multiple layers: document ingestion validation (authenticate sources, scan for adversarial patterns), and generation-time sandboxing (treat all retrieved content as untrusted data, not instructions). The combination of source trust and prompt-level content isolation is the effective defence.

Q13. You are building a RAG system for a medical Q&A application. The knowledge base contains conflicting information from different clinical guidelines published in different years. How should the system handle this?

- A. Return only the most recent document to avoid conflicts.
- B. Surface the conflict explicitly to the user: retrieve the conflicting sources, present both perspectives with their publication dates and source authorities, and note that guidelines differ — never silently choose one over the other for a medical decision.**
- C. Merge conflicting chunks into a single synthesised chunk during preprocessing.
- D. Use the LLM to determine which guideline is correct and return only that answer.

Correct Answer: B

Explanation

In medical contexts, silently resolving conflicts — whether by recency, authority, or LLM judgment — is dangerous. Clinical guidelines evolve and may conflict for legitimate reasons (different patient populations, emerging evidence). The correct approach is conflict detection and transparent presentation: the user (a clinician) must see both perspectives and decide. Silent conflict resolution removes critical information from the clinical decision-making process.

Q14. What is 'speculative RAG' and how does it reduce latency in generation-heavy pipelines?

- A. Speculatively pre-retrieving documents for likely future queries.
- B. Using a smaller, faster 'speculative' model to generate a draft answer from retrieved context while the larger model verifies and refines it in parallel — reducing wall-clock latency by overlapping generation and verification rather than running them sequentially.**
- C. Running retrieval and generation simultaneously without waiting for retrieval to complete.
- D. Pre-computing answers for frequently asked questions and serving them without live retrieval.

Correct Answer: B

Explanation

Speculative RAG applies speculative decoding principles to the RAG pipeline: a fast small model drafts the answer, a larger model verifies and corrects it. If the draft is accepted, the expensive large model is only used for verification (much cheaper than full generation). This parallelism reduces the latency of large-model RAG generation by 2–3× for queries where the small model's draft is mostly correct.

Q15. A production RAG system serves queries for both quick factual lookups ('What is our office address?') and deep analytical questions ('Summarise the trends in our Q1-Q4 2023 sales reports'). A single pipeline handles both. What is the main performance problem?

- A. Analytical questions require more retrieved chunks, slowing down simple queries.
- B. A single pipeline applies the same retrieval configuration (top-k, chunk size, reranking) to all queries — over-engineering simple queries (wasting latency and cost) and under-serving complex queries (insufficient context). A query complexity classifier routing to different pipeline configurations solves this.**
- C. Factual queries need BM25; analytical queries need dense retrieval.
- D. The LLM cannot switch between concise and analytical output modes.

Correct Answer: B

Explanation

One-size-fits-all RAG pipelines are inefficient across query complexity. A factual lookup needs top-1 retrieval, no reranking, a short answer prompt — very fast. An analytical query needs top-10 retrieval from multiple documents, reranking, long-form synthesis prompts — inherently slower. A query complexity router applies the right configuration to each query type, optimising cost and latency without sacrificing quality for either.

SET

5

Evaluation, Monitoring & Advanced RAG Patterns

Set 5 of 5 · 15 Questions · RAG & Knowledge Systems Question Bank

Q1. The RAGAS evaluation framework measures four metrics. Which combination gives the most complete picture of RAG system quality?

- A. Retrieval latency, generation latency, cost per query, uptime.
- B. Context precision (are retrieved chunks relevant?), context recall (is all necessary information retrieved?), answer faithfulness (are claims grounded in context?), and answer relevance (does the answer address the question?).**
- C. BLEU score, ROUGE score, BERTScore, and perplexity.
- D. Embedding model accuracy, reranker accuracy, generator accuracy, and end-to-end accuracy.

Correct Answer: B

Explanation

RAGAS's four metrics cover the full RAG pipeline: context precision and recall measure retrieval quality from two angles (precision = signal-to-noise in retrieved content; recall = completeness of retrieved information), answer faithfulness measures generation groundedness, and answer relevance measures whether the answer actually addresses the question. Together they pinpoint exactly which component is failing.

Q2. Your RAG system has high answer faithfulness (95%) but low answer relevance (60%). What does this indicate?

- A. The LLM is hallucinating frequently despite high faithfulness.
- B. The system is faithfully reporting what's in the retrieved context, but the retrieved context is not relevant to what the user actually asked — a retrieval failure, not a generation failure.**
- C. The embedding model needs to be retrained.
- D. The LLM is answering questions it wasn't asked.

Correct Answer: B

Explanation

High faithfulness + low relevance is a classic retrieval failure signature: the generator is doing its job (staying grounded) but retrieval is surfacing content that doesn't address the actual question. The answer is coherent and grounded but off-topic. The fix is in retrieval: query reformulation, better metadata filtering, or improved embedding alignment — not in the generation component.

Q3. What is 'online evaluation' in a RAG system and how does it complement offline evaluation?

- A. Running evaluation during document ingestion rather than at query time.
- B. Using real user feedback signals (thumbs up/down, follow-up questions, session abandonment, correction requests) to measure production quality — complementing offline evaluation on curated datasets by revealing distribution shift, long-tail failures, and real-world query patterns the offline set didn't anticipate.**
- C. Evaluating the system while it is actively serving queries to avoid downtime.
- D. Continuous model retraining based on production query logs.

Correct Answer: B

Explanation

Offline evaluation on curated datasets measures known failure modes. Online evaluation discovers unknown ones: real users ask questions the evaluation set didn't cover, phrase queries differently, and interact with the system in ways that reveal brittleness invisible in testing. Signals like 'did the user ask a follow-up suggesting the first answer was wrong?' are low-cost, high-signal online quality indicators.

Q4. A RAG system that performed well at launch degrades over 6 months despite no code changes. Monitoring shows answer quality declining steadily. Which combination of factors most likely explains this?

- A. The LLM provider degraded their model quality.
- B. A compound of knowledge base staleness (documents not updated), query distribution shift (users asking new types of questions), and index degradation (new document types added without adjusting chunking or embeddings) — each contributing to gradual quality decline.**
- C. The vector database's ANN index accuracy degrades over time.
- D. User expectations have increased, making the same quality appear worse.

Correct Answer: B

Explanation

RAG degradation over time is almost always multi-causal. Knowledge base staleness is the most common cause but rarely acts alone — query distribution shift (users' needs evolve) and index quality drift (new documents added with incompatible chunking) compound it. Monitoring must track all three: document freshness, query coverage, and index health — and trigger maintenance actions on each independently.

Q5. What is 'corrective RAG' (CRAG) and how does it improve robustness over standard RAG?

- A. A RAG system that corrects spelling errors in user queries before retrieval.
- B. A RAG pattern that evaluates the quality of retrieved documents using a lightweight relevance grader — if retrieved documents are rated low quality, the system falls back to web search or reformulates the query and retries before generating — reducing the risk of generating from poor-quality context.**
- C. A system that allows users to correct wrong answers and retrains the model.
- D. A post-generation step that corrects factual errors in the LLM's output.

Correct Answer: B

Explanation

CRAG adds a retrieval quality assessment step: after standard retrieval, a relevance grader scores the retrieved documents. If scores are too low, the system takes corrective action — web search fallback, query reformulation, or explicit 'insufficient information' response — before passing to generation. This prevents the classic RAG failure of generating plausible but fabricated answers from low-quality retrieval.

Q6. What is 'RAG fusion' and what advantage does it provide over single-query RAG?

- A. Fusing the outputs of multiple RAG systems into a single answer.
- B. Generating multiple variations of the user query (using an LLM), running retrieval independently for each, and fusing the result lists using Reciprocal Rank Fusion — improving recall by capturing documents that different query phrasings surface, at the cost of additional retrieval calls.**
- C. Combining dense and sparse retrieval results into a single ranked list.
- D. Merging multiple knowledge bases into a single unified index.

Correct Answer: B

Explanation

A single query embedding may miss relevant documents phrased differently than the user's exact wording. RAG fusion generates 3–5 query variations (paraphrases, different vocabulary, more specific/general) and retrieves for each. RRF merges the ranked lists, boosting documents that appear highly in multiple lists. This significantly improves recall on queries where the relevant document uses different terminology than the user.

Q7. What is 'adaptive retrieval' and when is it most valuable?

- A. Retrieval that adapts chunk size based on query length.
- B. A strategy where the RAG system decides dynamically whether to retrieve at all — for queries clearly answerable from LLM parametric knowledge without retrieval (e.g. general knowledge questions), skipping retrieval saves latency and cost while retrieval is reserved for domain-specific queries where the knowledge base adds value.**
- C. Retrieval that adapts the number of results based on the vector database's load.
- D. A system that learns from past queries to improve future retrieval rankings.

Correct Answer: B

Explanation

Retrieval has a cost (latency, compute, context tokens). For queries the LLM can answer correctly from parametric knowledge ('What is the capital of France?'), retrieval adds nothing. A classifier that determines 'does this query need knowledge base context?' routes retrieval-necessary queries to the full RAG pipeline and simple queries to direct LLM generation. This reduces median latency without degrading quality for domain-specific queries.

Q8. You want to measure whether a RAG system's answers are grounded without using an LLM judge (due to cost). Which approach provides a scalable alternative?

- A. Manual review of 10% of all answers.
- B. Use an NLI (Natural Language Inference) model to score entailment between each claim in the answer and the retrieved context — classifying each claim as entailed, contradicted, or neutral. Aggregate per-answer faithfulness scores across the pipeline automatically.**
- C. Compare answer embeddings to retrieved context embeddings using cosine similarity.
- D. Count the number of sentences in the answer that contain words from the retrieved context.

Correct Answer: B

Explanation

NLI-based faithfulness evaluation is the scalable alternative to LLM-as-judge. NLI models (e.g. DeBERTa-NLI) are small, fast, and specifically trained to classify textual entailment — much cheaper per inference than LLM judges. They score whether each generated claim is supported by the context. Cosine similarity measures topic overlap, not entailment; word-overlap counting misses paraphrases and catches false positives.

Q9. A RAG system is evaluated with a test set curated by domain experts. It achieves 92% accuracy. In production, user-reported quality is 71%. What evaluation design flaw explains the gap?

- A. The domain experts set unrealistically high standards.
- B. The expert-curated test set represents idealised, well-formed domain queries — not the noisy, ambiguous, conversational queries real users submit. The 21% gap reveals distribution mismatch: the evaluation measured performance on a population different from production.**
- C. The production LLM is a different version from the one used in evaluation.
- D. User-reported quality is subjective and less reliable than expert evaluation.

Correct Answer: B

Explanation

Expert-curated test sets are clean, complete, and precisely phrased — optimising for domain correctness but not for realistic query distribution. Real users write informally, incompletely, and ambiguously. A 21% accuracy gap is a strong signal that the evaluation set must include real user queries (sampled from production logs or user research) alongside expert queries. Both populations need to be represented for evaluation to predict production performance.

Q10. What is 'modular RAG' and what operational advantage does it provide over monolithic RAG pipelines?

- A. A RAG system divided into separate microservices for retrieval and generation.
- B. An architecture where each RAG component (query transformation, retrieval, reranking, context compression, generation) is a standalone, swappable module — allowing independent evaluation, replacement, and optimisation of each component without rebuilding the entire pipeline.**
- C. A RAG system that uses different LLMs for different document types.
- D. A pipeline that automatically selects the retrieval strategy based on the module configuration.

Correct Answer: B

Explanation

Modular RAG treats each pipeline stage as an independent unit with defined interfaces. This enables: A/B testing a new reranker without changing retrieval; replacing the embedding model without changing generation; adding query decomposition without touching context compression. Monolithic pipelines require testing and redeploying the entire system for each change. Modularity is the key operational advantage for iterating a production RAG system.

Q11. A RAG pipeline uses an LLM to evaluate its own retrieval quality ('Is this retrieved document relevant to the query?'). What is the key risk of this self-evaluation loop?

- A. The LLM evaluation call doubles the inference cost.
- B. The evaluator LLM may share the same biases as the generator LLM — rating documents as relevant that match the LLM's existing knowledge rather than the user's actual information need, creating a closed loop that reinforces the system's existing failure patterns.**
- C. Self-evaluation always returns high relevance scores regardless of document quality.
- D. The retrieved documents interfere with the LLM's ability to evaluate objectively.

Correct Answer: B

Explanation

When the same LLM family evaluates and generates, shared biases create correlated errors. If the LLM 'likes' a certain document style or topic framing, it will rate those documents as relevant AND generate answers aligned with them — creating a confirmation loop. Ideally, use a different model family for evaluation, or use NLI-based evaluation to introduce a truly independent signal.

Q12. What is 'long-context RAG' and what challenges does it introduce compared to standard top-k chunk retrieval?

- A. RAG over very long user queries requiring special query embedding.
- B. Passing a large amount of retrieved content (e.g. 50+ pages) to an LLM with a long context window instead of top-k chunks — enabling coverage of complex multi-document questions, but introducing lost-in-the-middle attention failures, higher cost, and longer latency compared to focused chunk retrieval.**
- C. A RAG approach where each document is processed in multiple sequential LLM calls.
- D. Retrieval from knowledge bases containing documents longer than 100,000 words.

Correct Answer: B

Explanation

Long-context RAG exploits large context windows (100k+ tokens) to pass many documents to the LLM at once, reducing retrieval precision requirements. Challenges: lost-in-the-middle (LLMs underweight content in the middle of long contexts), dramatically higher cost (long context = many tokens), and increased latency. For most queries, focused chunk retrieval with high precision outperforms long-context stuffing in both quality and efficiency.

Q13. A team measures their RAG system's 'context recall' metric at 0.45. What does this mean in practice?

- A. 45% of queries return at least one relevant document.
- B. On average, only 45% of the information needed to answer a query is present in the retrieved context — meaning the LLM is expected to fill the remaining 55% from parametric memory or produce incomplete answers, a strong signal that retrieval coverage is insufficient.**
- C. 45% of retrieved chunks are relevant to the query.
- D. The embedding model recalls 45% of documents correctly.

Correct Answer: B

Explanation

Context recall measures completeness: what fraction of the necessary information was retrieved? A score of 0.45 means more than half the information needed for a complete answer is absent from the retrieved context. At this level, either answers will be consistently incomplete or the LLM will hallucinate the missing information. The fix is in retrieval: increase top-k, improve chunking granularity, or expand the knowledge base coverage.

Q14. What is 'HyDE' (Hypothetical Document Embeddings) and what retrieval problem does it address?

- A. Embedding document summaries instead of full documents to reduce index size.
- B. Generating a hypothetical document that would answer the query using an LLM, embedding that hypothetical document, and using its embedding for retrieval — addressing the query-document embedding space mismatch where short queries embed differently from long relevant documents.**
- C. Generating synthetic queries for each document during indexing to improve retrieval.
- D. Using a larger embedding model (hyper-dimensional embeddings) to improve similarity matching.

Correct Answer: B

Explanation

HyDE addresses the fundamental asymmetry between query and document embeddings: a 10-word query occupies a different region of the embedding space than a 500-word document that answers it. By embedding a hypothetical full-answer document (written by the LLM) instead of the raw query, HyDE retrieves in the document-document similarity space rather than query-document space — significantly improving recall on complex, open-ended queries.

Q15. You are deploying RAG for an internal HR policy assistant. Employees ask about policies that may have changed. What monitoring metric most directly measures whether the knowledge base is keeping up with policy changes?

- A. Average retrieval latency per query.
- B. Document freshness gap — the average time between a policy document being officially updated and its new version being indexed and available in the knowledge base — directly measuring how stale the system's answers may be.**
- C. The number of new documents added to the knowledge base per month.
- D. User satisfaction scores on HR policy queries.

Correct Answer: B

Explanation

Document freshness gap directly measures the operational risk of outdated answers. If HR updates a parental leave policy on day 1 and the RAG index isn't updated until day 14, all queries about parental leave during that window receive wrong answers. Tracking time-from-official-update to time-in-index — and alerting when it exceeds a threshold (e.g. 24 hours) — is the most direct measure of knowledge currency for a policy assistant.

menler

Your turning point in the AI era.

END OF ASSESSMENT

*Your turning
point starts
here.*